LAB 4 – VIBECODING

Chat / LLM Interaction Log

Air Hockey – Iterative AI-Assisted Development

Student	Pratham P Shetty
Project	Air Hockey (Python / Pygame)
Lab	Lab 4 – VibeCoding
Repository	02_air_hockey
LLM Tool Used	ChatGPT

Documentation note: This document is a clean, formatted reconstruction of the LLM-assisted implementation workflow for submission documentation. It is not a raw export of a single chat transcript. The prompts and responses below describe the iterative development process and code-review steps used for the four assigned tasks.

1. Initial Project Analysis

Student Prompt: I have been assigned a Python/Pygame Air Hockey project for Lab 4 VibeCoding. The README specifies four tasks: fix puck–paddle collision, add match scoring, add a 30-second timer, and reset the puck after scoring. Analyze the existing project first and propose a minimal implementation plan without changing unrelated functionality.

LLM Response: The project should be modified incrementally. First inspect the game engine, puck, paddle, collision, and renderer logic. Implement and test collision handling before adding scoring. Then add the timer and final-match state. Finally, connect goal detection to a complete puck reset and immediate relaunch. Each task should be tested before moving to the next task.

2. Task 1 – Fix Puck–Paddle Collision

Student Prompt: The puck does not consistently respond correctly when it collides with a paddle, particularly at higher speeds or when it approaches at an angle. Fix the collision logic so the puck reliably rebounds from paddles at different speeds and angles without passing through or getting stuck inside them. Keep the existing game controls and visuals.

LLM Response: Use geometric circle-to-circle collision detection rather than relying only on a simple axis check. Calculate the collision normal from the paddle center to the puck center, move the puck outside the paddle by the overlap amount, and reflect the puck velocity using the collision normal. To reduce tunneling at higher speeds, update the physics in small substeps.

Student Review / Test: I tested paddle hits at different approach directions. The puck rebounded more reliably, but a fast puck could still move too far between frames.

LLM Follow-up: Increase physics stability by using multiple smaller simulation steps per frame and perform collision detection during each substep. Keep the final puck position corrected so it cannot remain embedded in a paddle.

Result: Task 1 was retested with straight and angled impacts. The collision response was stable and the puck was prevented from remaining inside the paddle.

3. Task 2 – Implement Match Scoring

Student Prompt: Implement match scoring. A player earns one point when the puck enters the opponent's goal. Display both scores during gameplay and determine the winner from the final score. Do not break the collision behavior from Task 1.

LLM Response: Maintain separate player and computer score variables in the game engine. Detect a goal only when the puck crosses the appropriate goal boundary. Increment the corresponding score and render both scores at the top of the screen. At match end, compare the two scores and display the winner or a draw.

Student Review / Test: I tested goals on both sides and confirmed that the appropriate score increased. I also checked that hitting the normal wall area did not incorrectly award a point.

Result: Task 2 was implemented and tested. Both scores are displayed and the final result is determined from the scores.

4. Task 3 – Implement 30-Second Match Timer

Student Prompt: Add an exact 30-second countdown timer. Display the remaining time during gameplay. When it reaches zero, stop the match and determine the result from the scores. If the scores are equal, display Draw.

LLM Response: Record the match start time and calculate elapsed time using Pygame's timing functions rather than decrementing a counter once per frame. Compute remaining time as 30 seconds minus elapsed time, clamp it to zero, and end the match when it reaches zero. At that point, freeze gameplay and display PLAYER WINS!, COMPUTER WINS!, or DRAW.

Student Review / Test: I ran the game through the end of the timer. The display reached 00 and the game showed the appropriate winner based on the current score.

Result: Task 3 was completed. The match ends at 30 seconds and the final result is shown.

5. Task 4 – Reset Puck After Scoring

Student Prompt: After every goal, reset the puck to the center of the table and prepare it for the next point. Reset the velocity, direction, and any relevant state so that the next point starts cleanly. The puck must not remain motionless after a goal.

LLM Response: Create a dedicated puck-reset routine. Set the puck position to the exact center of the playfield, clear its previous velocity, reset any goal-related state, and immediately assign a new launch velocity. Call this routine only after a goal, not when the 30-second match ends.

Student Review / Test: I tested the match after a goal and checked that scoring, puck positioning, and the next-point launch were independent of the previous puck velocity.

LLM Follow-up: Make the reset explicit by setting both velocity components to zero before assigning the new launch direction. This prevents stale momentum from the previous point from carrying over.

Result: Task 4 was implemented so that a goal returns the puck to the center and starts the next point. The timer-expiry state remains separate, so the puck is not reset when the match has already ended.

6. Final Integration Test

- ✓ Puck rebounds from paddles at straight and angled approaches.
- ✓ Fast puck movement does not normally tunnel through paddles.
- ✓ Player and computer scores update after valid goals.
- ✓ Normal wall impacts do not award a goal.
- ✓ Timer starts at 30 seconds and reaches 00 at match end.
- ✓ Match stops when the timer expires.
- ✓ Higher score produces the corresponding winner message.
- ✓ Equal scores produce DRAW.
- ✓ After a goal, puck returns to the center and immediately launches.
- ✓ R key restarts the match and resets the match state.

7. Final Implementation Summary

The Air Hockey project was developed through an iterative VibeCoding workflow. Each requested feature was approached separately, tested, reviewed, and then integrated with the existing game. The final implementation preserves the original Pygame structure while adding collision robustness, scoring, a 30-second match timer, end-of-match results, and a clean puck reset after every goal.